SEMESTER 1 2024/2025

FACULTY:

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY (FTMK)

COURSE:

BITI 2223

MACHINE LEARNING

CASE STUDY

GROUP : S1G1 G1

TITLE :

REINFORCEMENT LEARNING CASE STUDY

LECTURER : Assoc. Prof. Dr. Choo Yun Huoy

Table of Content

Index	Content
1	Main Title
2	Table of Content
3-4	First Question Formal definition of Reinforcement Learning, including the basic concepts and terminology.
5-15	Second Question (4) commonly used reinforcement learning techniques, (2) from the conventional machine learning and (2) from the deep learning approaches.
16-31	Third Question - First Example Smart Grid Energy Management
32-42	Third Question - Second Example Autonomous Warehouse Robot Navigation
43-44	Forth Question Future direction of Reinforcement Learning trends and evolution in the next five years **in Malaysia industry** with (5) potential key companies in this field.

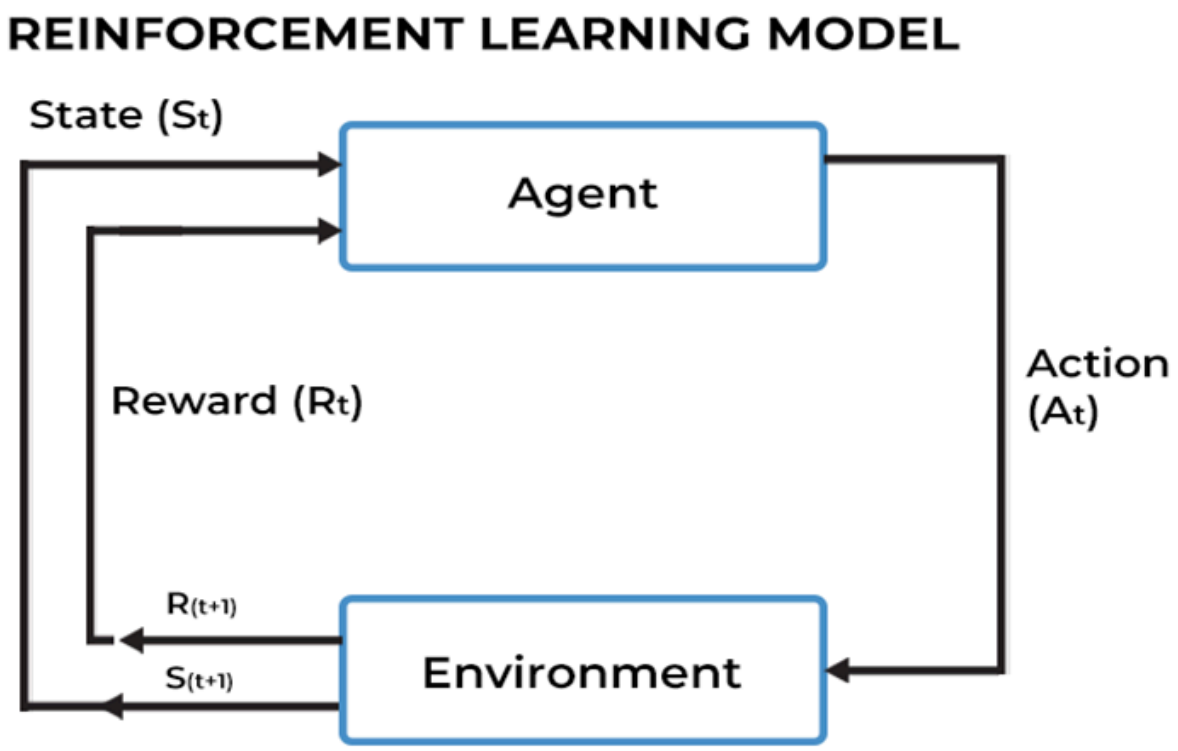
BITI 2223 MACHINE LEARNING – REINFORCEMENT LEARNING

1. Formal Definition of Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make sequential decisions by interacting with an environment. The agent seeks to maximize cumulative rewards over time by taking actions based on observed states and receiving feedback in the form of rewards or penalties. Unlike supervised learning, RL does not rely on labeled datasets but uses a trial-and-error approach to discover optimal actions.

Key Concepts of Reinforcement Learning

- **Agent:** The learner or decision-maker.
- **Environment:** Everything the agent interacts with.
- **State:** A specific situation in which the agent finds itself.
- **Action:** All possible moves the agent can make.
- **Reward:** Feedback from the environment based on the action taken.



Reinforcement Learning ;

Reinforcement Learning (RL) has seen significant advances across various complex decision-making tasks. However, deploying RL in real-world scenarios such as autonomous driving and robotics raises safety concerns. This has led to an increased focus on developing safe RL algorithms. Despite the long history of safe control methods, the exploration of safe RL algorithms is relatively nascent. This paper provides a comprehensive review of safe RL, focusing on its methods, theories, and practical applications. We begin by outlining the progress in safe RL across five dimensions and identify five key challenges, termed "2H3W," (2 “Human factors " 3"Workflows") that need addressing for effective real-world deployment of safe RL. We then delve into the algorithmic and theoretical advancements in addressing these challenges, including a discussion on the sample complexity of safe RL algorithms. We also present an overview of applications and benchmarks for safe RL. In conclusion, we discuss the ongoing challenges in the field, aiming to foster further research. Additionally, we offer an open-source repository that includes major safe RL algorithms, accessible via the provided link. This paper aims to lay a solid foundation for future research in safe RL, contributing to safer applications in critical areas.

Reinforcement Learning Techniques:

In the context of reinforcement learning (RL), the commonly used techniques are – Q-Learning, SARSA, Deep Q-Network (DQN), and Proximal Policy Optimization (PPO) which can be categorized into two groups: conventional machine learning approaches and deep learning approaches.

Conventional Machine Learning Approaches:

These approaches typically do not inherently involve neural networks or deep learning architectures but can be extended to use them.

Q-Learning:

Q-Learning: Introduction

Q-Learning is a model-free reinforcement learning algorithm that enables an agent to learn an optimal policy for selecting actions by updating Q-values. These Q-values represent the expected rewards for actions taken in specific states. This algorithm doesn't require prior knowledge of the environment and adjusts itself based on real-time interactions.

Key Features of Q-Learning

1. Q-Values or Action-Values:
   Q-values are estimates of the potential reward from performing an action in a given state, denoted as Q(S,A)Q(S, A)Q(S,A). These values are updated iteratively using the Temporal Difference (TD) formula.
2. Rewards and Episodes:
   The agent interacts with its environment in episodes. It transitions between states by taking actions and receiving rewards, eventually reaching terminal states where an episode ends.
3. Temporal Difference (TD) Update Rule:
   The Q-value is updated as:

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

Where:

- S: Current State of the agent.
- A: Current Action Picked according to some policy.
- S': Next State where the agent ends up.

- A': Next best action to be picked using current Q-value estimation, i.e. pick the action with the maximum Q-value in the next state.
- R: Current Reward observed from the environment in Response of current action.
- γ (>0 and <=1) : Discounting Factor for Future Rewards. Future rewards are less valuable than current rewards so they must be discounted. Since Q-value is an estimation of expected rewards from a state, discounting rule applies here as well.
- α: Step length taken to update the estimation of Q(S, A).

Action Selection (ϵ-Greedy Policy):

This policy balances exploration and exploitation:

- With probability $1-\epsilon$, the agent chooses the action with the highest Q-value.
- With probability ϵ, it explores by selecting a random action.

Implementation Process of Q-Learning

1. **Environment Definition:**
   Set up parameters such as states, actions, and a goal. For the taxi system, states represent grid positions, and actions correspond to movement directions (e.g., north, south).
2. **Hyperparameter Setup:**
   Specify learning rate (α\alpha), discount factor (γ\gamma), exploration rate (ϵ\epsilon), and training iterations.
3. **Algorithm Execution:**
   The agent starts from a random state, selects actions, transitions to new states, and updates Q values based on rewards. This process repeats until it learns optimal routes.

4. **Q-Table Output:**
   After training, the Q-table reflects learned action-reward associations for all state-action pairs.

Advantages of Q-Learning

- Learns long-term optimal strategies through iterative updates.
- Corrects mistakes during training, minimizing future errors.
- Requires no model of the environment, making it versatile.

Disadvantages of Q-Learning

- Inefficient with large state-action spaces due to the dimensionality of the Q table.
- Real-world applications like robotics can be costly due to hardware maintenance.

Applications of Q-Learning

1. **Game AI:** Achieves superhuman performance in Atari games.
2. **Robotics:** Enables navigation and obstacle avoidance.
3. **Traffic Optimization:** Manages autonomous vehicle routing and signal timings.
4. **Healthcare:** Personalizes treatment plans based on patient data.
5. **Finance:** Enhances algorithmic trading strategies.

- **Recent Development:**
  - The application of Q Learning has extended into various domains, including robotics and autonomous systems, where its ability to learn optimal policies without requiring a model of the environment is crucial.

- A recent significant advancement in Q-Learning is highlighted in the study "Parallel Q-Learning: Scaling Off-policy Reinforcement Learning under Massively Parallel Simulation" by Zechu Li et al., presented at ICML 2023. This work addresses scalability and efficiency, crucial for real-world applications

- **Reference:.**
  - geeksforgeeks.org/q-learning-in-python/
  - ICML 2023 Paper on Parallel Q-Learning

2. SARSA:

State-action-reward-state-action (SARSA) is an on-policy reinforcement learning algorithm used to train machine learning models for decision-making tasks modeled as Markov decision processes. In this method, the algorithm navigates from one state (S) to another (S1) by performing an action (A) and receiving a corresponding reward (R), followed by another action (A1) in the new state. This sequence forms the SARSA tuple (S, A, R, S1, A1).

Understanding SARSA: SARSA, an on-policy reinforcement learning algorithm, trains a Markov decision process model through a series of actions and states, culminating in a tuple that encapsulates these elements: state, action, reward, subsequent state, and subsequent action.

SARSA Algorithm Overview: Differing from Q-learning, SARSA updates its Q-values considering the action actually taken in the next state, rather than the optimal action. This on policy approach integrates the reality of the agent's actual policy into its learning process, potentially resulting in more robust policy learning under certain conditions.

How SARSA Operates: The SARSA algorithm updates a Q-table, which holds the values for state-action pairs estimating the value of actions taken in given

states. This process begins with initializing Q-values and selecting actions using an epsilon-greedy policy, balancing exploration (discovering new actions) and exploitation (utilizing known actions). Actions are selected, rewards are observed, and Q-values are updated iteratively. This cycle repeats until a satisfactory policy is developed, typically assessed over numerous episodes or until a predefined convergence criterion is met.

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

Comparison with Q-Learning: While Q-learning seeks to learn the optimal policy indirectly by always considering the best possible future action, SARSA directly incorporates the actual policy's actions into its updates, reflecting a fundamental difference in their approach to learning: SARSA is on-policy, Q-learning is off-policy.

Practical Application: Implementing SARSA might involve setting up an environment such as the FrozenLake from OpenAI Gym, where the algorithm learns to navigate a grid by making safe steps. Actions are taken based on current policy decisions, and learning adjusts based on the outcome of these actions.

This approach highlights SARSA's utility in environments where safety and adherence to a specific policy are crucial, such as autonomous navigation tasks where predictable behavior based on learned policies is necessary for safe operation.

- **Recent Development:**
    - SARSA continues to be relevant, particularly in scenarios requiring policy adherence, such as navigational tasks where safety and reliability are paramount.

- The integration of SARSA with deep learning frameworks has been explored to enhance its performance in complex decision environments. This melding of techniques helps in environments with high-dimensional state spaces.

- **Reference:**
  - builtin.com/machine-learning/sarsa

Deep Learning Approaches:

These methods integrate deep neural networks into their frameworks, making them suitable for handling high-dimensional or complex data inputs

1. Deep Q Network (DQN):

The DQN algorithm employs a deep learning framework to refine action-value functions. DQN leverages the Q-learning algorithm, which aims to estimate the optimal action-value function (Q-function) that maps states to expected future rewards.

- **State Representation:** Transforms the current environmental state into a numerical format, like raw pixel data or processed features, suitable for processing.
- **Neural Network Architecture:** Utilizes a deep neural network, often a convolutional neural network (CNN), which inputs the state and outputs action-values for each potential action.
- **_Experience Replay:** Saves the agent's experiences, which include the state, action, reward, and subsequent state, in a memory buffer designated for replay.
- **Q Learning Update:** Draws mini-batches of experiences from the memory to adjust the neural network's weights. This update uses a loss function based on the Bellman equation to reduce the difference between predicted and actual action-values.

- **Exploration and Exploitation:** Maintains a balance between using the current policy to choose actions greedily and opting for actions randomly to foster exploration.
- **Target Network:** Employs a separate but identically structured target network to enhance the stability of the learning process. This network's weights are periodically synced with the main network.
- **Iterative Process:** Continuously interacts with the environment, collects experiences, updates the network, and refines the policy until the desired level of performance is achieved.

Advantages:

- **Deep Representation Learning:** DQN utilizes deep neural networks to master complex and high-dimensional state representations, which enhances its ability to operate effectively in intricate environments.
- **Sample Efficiency:** Techniques like experience replay and target networks boost DQN's sample efficiency by enabling the reuse and reduction of correlation in experiences.
- **Generalization:** DQN is adept at generalizing its learned policies to new, unseen states, which enhances its adaptability and decision-making in unfamiliar situations.

Limitations:

- **Hyperparameter Sensitivity:** The success of DQN heavily relies on the precise adjustment of hyperparameters such as the learning rate, exploration rate, and network architecture, which necessitates meticulous tuning.
- **Lack of Continual Learning:** Predominantly tailored for offline batch learning, DQN does not efficiently support scenarios that require online continual learning.

- **Overestimation of Action-Values:** The Q-learning update mechanism in DQN may result in the overestimation of action values, which can adversely affect the precision of the policy it learns.

- **Recent Development:**
    - DQN's architecture has been enhanced with improvements like dueling network architectures and prioritized experience replay, which help in stabilizing the learning process and improving convergence.
    - The paper "Dueling Network Architectures for Deep Reinforcement Learning" by Ziyu Wang et al. delves into these advancements, offering significant insights into how modifying the network architecture can lead to substantial performance gains
- **Reference:**

Dhumne, S. (2023). Deep Q-Network (DQN). Medium.

Dueling Network Architectures for Deep Reinforcement Learning,Ziyu Wang.

2. Proximal Policy Optimization (PPO):

_Proximal Policy Optimization (PPO) is a reinforcement learning algorithm designed to train computer agents to perform complex tasks efficiently. Developed by John Schulman in 2017, PPO has become a standard algorithm at OpenAI due to its effectiveness and practicality.

PPO is praised for its simplicity, stability, and efficiency in utilizing samples. It's categorized as a policy gradient method, which involves minor policy updates to ensure consistent progression towards optimal solutions. By

implementing a clipping mechanism, PPO limits the size of policy updates, preventing drastic changes that could derail the learning process.

What is PPO?

_In reinforcement learning, a policy maps actions to states, guiding the agent's decisions based on its current environment. PPO helps evaluate and enhance this policy by calculating gradients, which adjust the policy towards more successful strategies.

Operational Mechanism of PPO: Unlike traditional methods, PPO modifies the policy by considering both the new and old policy parameters, focusing on maintaining a balance between exploration (trying new actions) and exploitation (leveraging known actions).

$$L^{clip}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot A_t)\right]$$

$$, r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

Where:

st = (Current State)

at =(Action):The action taken by the agent in state sts_tst.

st = (Next State)

At = (Advantage Function):

rt(θ) = (Probability Ratio).

ϵ\epsilon = (Clipping Parameter):

α\alpha = (Learning Rate):

γ\gamma = (Discount Factor):

PPO's Unique Features:

- **Clipped Surrogate Objective:** PPO utilizes a clipped surrogate objective to control the extent of policy updates, ensuring they remain within a predefined range to avoid destabilizing the learning process.
- **Iterative Updates:** It employs multiple epochs and mini-batch updates during training, using trajectories collected from the environment to optimize the policy.
- **Value Function Estimation:** To reduce variance and stabilize training, PPO integrates value function estimation, enhancing the prediction of cumulative rewards.
- **Parallel Training Capability:** PPO supports parallel training across multiple environments, accelerating the learning process.

Architecture and Application: PPO's architecture typically involves separate networks for the policy and value functions, often using neural networks like CN Implementing PPO involves setting up an environment, initializing the policy and value networks, and iteratively adjusting these networks based on the experiences gathered during agent-environment interactions. This process is facilitated by TensorFlow or similar frameworks that support efficient computation of gradients and network training.
Ns for complex state spaces or RNNs for sequential data. The networks are trained using loss functions tailored to maximize the policy's performance while stabilizing updates through clipping.

Example Application - Navigating a Grid World: Consider an agent in a grid world where it must reach a goal while avoiding obstacles. PPO trains the agent to navigate by maximizing rewards for reaching the goal and minimizing penalties for hitting obstacles. The training involves adjusting policy parameters

based on the collected state-action-reward sequences, continuously refining the strategy through the PPO algorithm.

Implementation Insight:

Recent Development:

- PPO remains a popular choice for training policies in environments where exact simulations are costly. Its simpler objective and more stable updates make it suitable for a broad range of applications.
- John Schulman's work on PPO, which outlines its effectiveness and the mathematical underpinnings that contribute to its robust performance, is a critical read for understanding its impact and applications

Reference:

DhanushKumar, D. (2023). PPO Algorithm. Medium.

John Schulman's work

3. Real-World Use Case of Reinforcement Learning

First Example: Smart Grid Energy Management

(i) Use Case Description

In smart grids, reinforcement learning can optimize energy distribution and consumption by dynamically adjusting the power supply based on demand and generation patterns.

(ii) Solution Proposal

- Solution Methodology:
  - **Environment Setup:**
    - Simulate a smart grid environment with multiple energy sources (solar, wind, etc.) and consumers (households, industries, etc.).
  - **Q-Learning Basics:**
    - States (S):
      - State 0: Energy consumption level of households
      - State 1: Energy consumption level of industrials
      - State 2: Overall grid load or remaining capacity
      - State 3: Energy output from solar sources
      - State 4: Energy output from wind sources
      - State 5: Battery storage level or output
    - Actions (A):
      - Action 0: Adjust energy output from solar sources
      - Action 1: Adjust energy output from wind sources
      - Action 2: Adjust energy output from battery storage
    - There's **NO GOAL STATE** in the smart grid energy management case. Since the task is to balance energy consumption and production dynamically, which is a continuous optimization problem rather than a discrete goal-reaching task.
    - Rewards (R) : The reward is calculated dynamically based on state and action.
      - State-specific rewards:
        - $R_0 = - | S_0 - S_3 |$
        - $R_1 = - | S_1 - S_4 |$

- $R_2 = - | S_2 | - 1(S_1 < 0.5)$
- $R_3 = S_3 - | S_0 - S_3 |$
- $R_4 = S_s - | S_1 - S_4 |$
- $R_5 = S_5 - | (S_0 - S_1) + ((S_3 - S_4 + S_5) |$

- Action-specific rewards:
  - $A_0 = S_3$
  - $A_1 = S_4$
  - $A_2 = S_5$
- Total rewards:
  - $R = R_0 + R_1 + R_2 + R_3 + R_4 + R_5 + R_{Action}$

➢ Q-Table: A table mapping state-action pairs to expected rewards.

❖ **Algorithm steps (Step-by-step description):**

➢ Initializing the Q-table with zeros.

➢ Setup the hyperparameter:

- **Learning rate (α)**: 0.1.
- **Discount factor (γ)**: 0.99
- **Exploration rate**: Initialized at 1.0 (pure exploration), and decays over time.
- **Exploration decay rate**: 0.05
- **Number of episodes**: 5000
- **Max steps per episodes**: 1000

➢ Each episode, the agent starts with a random initial state. (Simulates an unpredictable environment)

➢ Discretize the State using bins, help mapping the state to a discrete value.

➢ The agent either explores new actions or exploits the best-known action based on the current Q-values using epsilon-greedy strategy.

➢ The agent takes the selected action and the environment responds with a new state and a reward.

➢ After taking the action, the agent moves to the new state. This state is also discretized to maintain consistency in indexing the Q-table.

- The agent updates the Q-value for the state-action pair using the **Bellman equation**.

reward discount factor

$$Q(s,a) = r(s,a) + \gamma \max_{a} Q(s',a)$$

state action

- After each episode, the exploration rate is decayed.
- Repeat for all episodes (which is 5000 in the hyperparameter).

- Illustration:

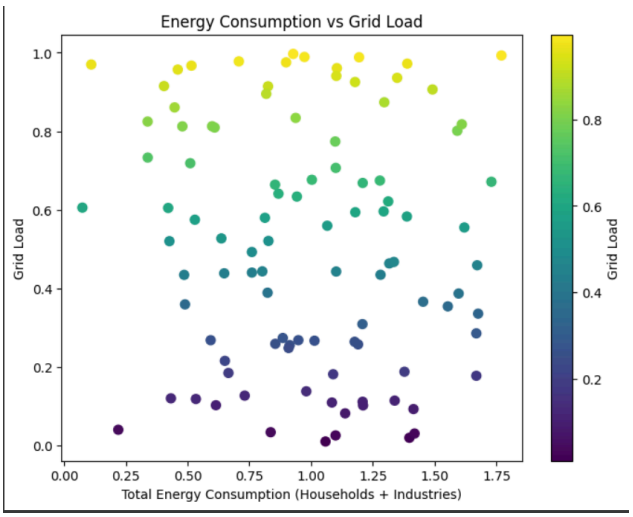
Since in this case we have 6 states and 3 actions, and we discretize into 4 bins ($4^6 = 4096$ combination of states) the Q-table (initiate with zeros) is:

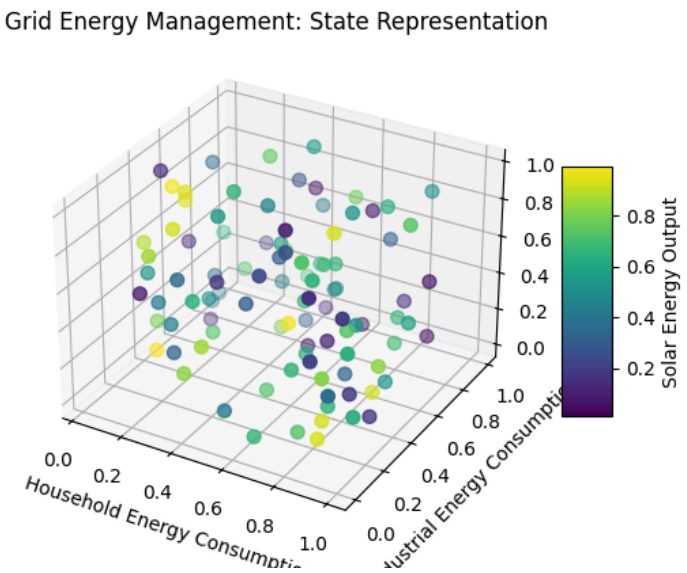
State	Action 0	Action 1	Action 2
(0,0,0,0,0,0)	0	0	0
(0,0,0,0,0,1)	0	0	0
(0,0,0,0,1,0)	0	0	0
...	...	...	...
(3,3,3,3,3,3)	0	0	0

And for the R-matrix (Assuming State (0, 1, 0, 0, 1, 0)) is:

State	Action 0	Action 1	Action 2
(0,0,0,0,0,0)	0	0	0
(0,0,0,0,0,1)	0	0	0
(0,0,0,0,1,0)	0	1	-1
(1,1,1,0,1,1)	-1	0	1
...	...	...	...
(3,3,3,3,3,3)	-1	1	0

Since states have 4096 combinations, hard to illustrate a x * y grid layout, hence:





(iii) **<u>Commercialization Value and Stakeholders</u>**

Commercialization Value

1. Cost Efficiency & Savings:
    - The Smart Grid Energy Management system can help industries, households, and utilities optimize their energy consumption by balancing energy demand and supply efficiently. By integrating renewable sources (solar, wind) and storage (batteries), the system can significantly reduce energy costs by avoiding reliance on expensive, non-renewable power sources during peak demand.
    - Potential Savings: This can translate to cost savings for energy producers and consumers by automating energy distribution, reducing reliance on the grid, and minimizing waste from overproduction.
    - Dynamic Pricing: The system's ability to predict demand and optimize energy distribution can also enable dynamic pricing models, which can further enhance cost efficiency.
2. Grid Stability & Reliability:
    - By efficiently managing energy generation and consumption, the system contributes to a more reliable and resilient energy grid. The ability to shift energy loads and balance distributed generation can prevent grid failures and outages, reducing the costs associated with infrastructure damage and recovery.
    - Renewable Integration: Facilitates better integration of renewable energy sources into the grid, which is critical for countries or regions aiming to reduce their carbon footprint and dependency on fossil fuels.
3. Sustainability & Green Initiatives:
    - The technology promotes sustainable energy practices by utilizing renewable sources and battery storage, which can attract organizations and governments aiming to meet environmental regulations and reduce carbon emissions.
    - Environmental Regulations: With increasing regulatory pressure on businesses and industries to meet environmental goals (such as reducing

emissions), this system offers a compliance solution while improving energy efficiency.

Target Stakeholders

1. Energy Producers (Utility Companies):
    - Utility companies that generate, distribute, and sell energy are the primary stakeholders who can benefit from the Smart Grid system. By optimizing grid operations, reducing waste, and integrating renewable sources, they can cut costs and improve their bottom line.
    - Benefit: Reduced energy production costs, better load management, fewer system failures, integration with renewable energy sources.
2. Commercial and Industrial Consumers:
    - Large companies with high energy demands (e.g., factories, data centers, office buildings) can significantly benefit from the system's ability to balance supply and demand.
    - Benefit: Cost savings through optimized energy use, reduced electricity bills, improved sustainability, and corporate social responsibility (CSR) credentials.
3. Households (Residential Consumers):
    - Residential consumers, particularly those with solar panels or battery storage, can use the system to optimize their energy use, reduce dependence on the grid, and lower energy bills.
    - Benefit: Reduced electricity costs, enhanced energy independence, and an opportunity to support environmental sustainability.
4. Government & Regulators:
    - Government entities and regulatory bodies are stakeholders interested in policies for energy efficiency, sustainability, and grid resilience. They can play a role in incentivizing and promoting smart grid systems through subsidies, tax breaks, and regulatory support.

- Benefit: Achieving environmental goals, ensuring grid reliability, and enhancing overall energy infrastructure.

5. Technology Providers (IoT, AI, and Cloud Solution Providers):
    - Companies providing the hardware and software solutions (such as IoT devices, AI algorithms, cloud infrastructure) can find value in collaborating to create an integrated smart grid solution.
    - Benefit: Expanding their product/service offerings, revenue from licensing AI models, or providing technology solutions for smart grid implementations.

Import the neccessary libraries

```
[ ] #Install gym library
    !pip install gym
```

```
Requirement already satisfied: gym in /usr/local/lib/python3.10/dist-packages (0.25.2)
Requirement already satisfied: numpy>=1.18.0 in /usr/local/lib/python3.10/dist-packages (from gym) (1.26.4)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from gym) (3.1.0)
Requirement already satisfied: gym_notices>=0.0.4 in /usr/local/lib/python3.10/dist-packages (from gym) (0.0.8)
```

```
[ ] # Import the necessary libraries
    import gym
    import numpy as np
```

Initialize environment parameters

We define that in this Smart Grid Energy Environment, it consists of 3 action (Energy Sources) and 6 states (Consumers), where:

```
Action 0: Adjust energy output from solar sources
Action 1: Adjust energy output from wind sources
Action 2: Adjust energy output from battery storage source
State 0: Energy consumption level of households
State 1: Energy consumption level of industries
State 2: Overall grid load or remaining capacity
State 3: Energy output from solar sources.
State 4: Energy output from wind sources.
State 5: Battery storage level or output.

State 5 represents the battery storage level, which is crucial for maintaining a balanced grid.

Knowing the battery levels allows the agent to optimize its usage, especially during peak demand or low renewable energy periods.
```

There's **NO GOAL STATE** in Smart Grid Energy Management case.

Since the task is to balance energy consumption and production dynamically, which is a continuous optimization problem rather than a discrete goal-reaching task.

Calculate reward dynamically based on state and action.

The traditional obstacles does not apply in this environment, instead we try to make constraints like penalties for grid instability (low state[2]) serve a similar purpose.

Penalty for grid instability, because when the overall grid load or remaining capacity is too low, it indicates that the grid is running close to its maximum capacity, which can lead to instability.

This instability might manifest as power outages, voltage fluctuations, or overloading of grid components.

Action-Specific Reward:

```
Each action is tied to a specific state parameter:
Action 0: Rewards higher output from solar sources (state[3]).
Action 1: Rewards higher output from wind sources (state[4]).
Action 2: Rewards efficient use of battery storage (state[5]).
```

State-Specific Reward:

```
Each state is tied to a specific parameter:
Reward 0: Household consumption vs. solar (-abs(state[0] - state[3])
Reward 1: Industrial consumption vs. wind (-abs(state[1] - state[4])
Reward 2: Grid load penalty (-abs(state[2]) - (1 if state[2] < 0.5 else 0))
Reward 3: Soloar output (state[3] - abs(state[0] - state[3]))
Reward 4: Wind output (state[4] - abs(state[1] - state[4]))
Reward 5: Battery output (state[5] - abs((state[0] + state[1]))
```

```python
def calculate_reward(state, action):
    # State-specific rewards
    R_0 = -abs(state[0] - state[3])  # Household consumption vs. solar
    R_1 = -abs(state[1] - state[4])  # Industrial consumption vs. wind
    R_2 = -abs(state[2]) - (1 if state[2] < 0.5 else 0)  # Grid load penalty
    R_3 = state[3] - abs(state[0] - state[3])  # Solar output
    R_4 = state[4] - abs(state[1] - state[4])  # Wind output
    R_5 = state[5] - abs((state[0] + state[1]) - (state[3] + state[4] + state[5]))  # Battery output

    # Add action-specific incentives
    action_reward = 0
    if action == 0:  # Adjust solar output
        action_reward = state[3]
    elif action == 1:  # Adjust wind output
        action_reward = state[4]
    elif action == 2:  # Adjust battery output
        action_reward = state[5]

    # Combine all rewards
    reward = R_0 + R_1 + R_2 + R_3 + R_4 + R_5 + action_reward
    return reward
```

Take a step in the environment.

In this environment, we randomly determines if the episode is done with a 10% chance.

This is for simulates an unpredictable environment, adding an element of randomness and making the agent's learning process more robust.

It ensures the agent doesn't rely on a fixed episode length and can handle unexpected terminations effectively.

```
[ ] def step_environment(state, action, observation_space):
        reward = calculate_reward(state, action)
        new_state = np.random.rand(6)
        done = np.random.random() < 0.1 # Randomly determines if the episode is done with a 10% chance.
        return new_state, reward, done
```

Discretize the state space

Convert the continuous state space into discrete bins.

Continuous state spaces can have an infinite number of possible values. Discretizing them makes the problem more manageable by reducing the number of states.

This is important since Q-learning uses a Q-table to store the value of each state-action pair. A continuous state space would require an impractically large table, but a discrete space allows for a fixed-size table.

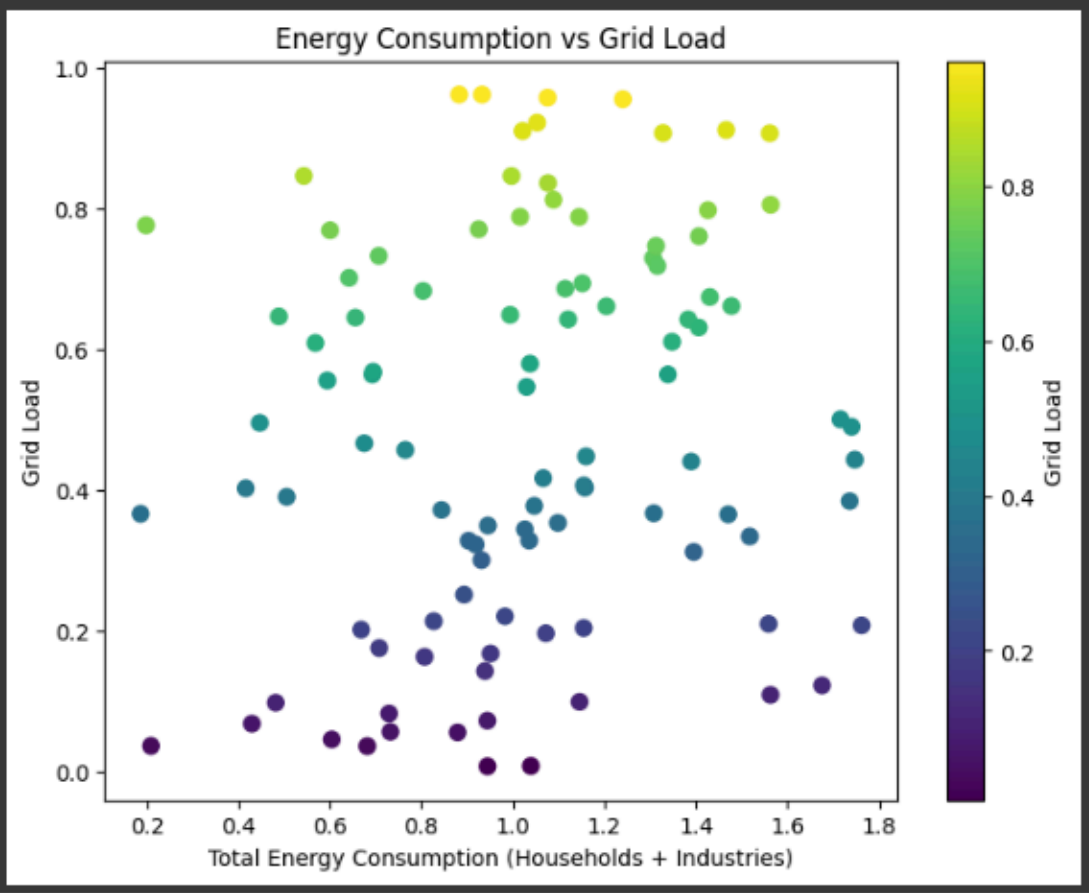
```
[ ] # Function to discretize the state space
    def discretize_state(state, bins):
        discrete_state = [min(len(bins) - 1, np.digitize(state[i], bins[i]) - 1) for i in range(len(state))]
        return tuple(discrete_state)
```

Define bins to discretize the state space

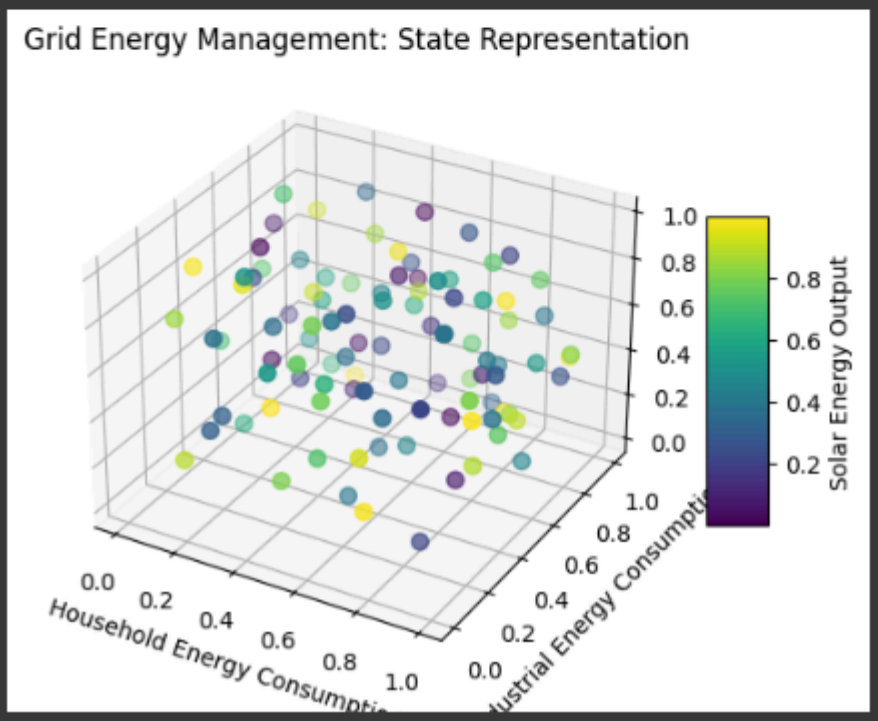
```
[ ] state_bins = [np.linspace(0, 1, 4) for _ in range(observation_space.shape[0])]
    state_size = tuple(len(bins) for bins in state_bins)
    action_size = action_space.n
    q_table = np.zeros(state_size + (action_size,)) # Initialize Q-table with all zero
```

Visualize the Grid Layout

```python
import matplotlib.pyplot as plt
import numpy as np

# Example state data (for plotting)
state_0 = np.random.rand(100)  # Household energy consumption
state_1 = np.random.rand(100)  # Industrial energy consumption
state_2 = np.random.rand(100)  # Grid load (remaining capacity)

# Combine household and industrial consumption for total consumption
total_consumption = state_0 + state_1

# Create a 2D plot of total energy consumption vs grid load
plt.figure(figsize=(8, 6))
plt.scatter(total_consumption, state_2, c=state_2, cmap='viridis', s=50)

# Labels and title
plt.xlabel('Total Energy Consumption (Households + Industries)')
plt.ylabel('Grid Load')
plt.title('Energy Consumption vs Grid Load')

# Add a color bar to represent grid load levels
cbar = plt.colorbar()
cbar.set_label('Grid Load')

plt.show()
```



```python
import matplotlib.pyplot as plt
import numpy as np

# Example state data
state_0 = np.random.rand(100)  # Household energy consumption
state_1 = np.random.rand(100)  # Industrial energy consumption
state_2 = np.random.rand(100)  # Grid load (remaining capacity)
state_3 = np.random.rand(100)  # Solar energy output
state_4 = np.random.rand(100)  # Wind energy output
state_5 = np.random.rand(100)  # Battery storage level

# Create a 3D scatter plot of State 0, State 1, and State 2
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Scatter plot with color mapping based on solar output (state 3)
scatter = ax.scatter(state_0, state_1, state_2, c=state_3, cmap='viridis', s=50)

# Labels and title
ax.set_xlabel('Household Energy Consumption')
ax.set_ylabel('Industrial Energy Consumption')
ax.set_zlabel('Grid Load')
ax.set_title('Grid Energy Management: State Representation')

# Color bar
cbar = fig.colorbar(scatter, ax=ax, shrink=0.5, aspect=5)
cbar.set_label('Solar Energy Output')

plt.show()
```



The reward table

```python
# Create a reward table to store rewards for all state-action combinations
reward_table = np.zeros(state_size + (action_size,))

# Example state to calculate the reward table
# Let's use a fixed state for demonstration purposes
example_state = np.array([0.4, 0.6, 0.3, 0.5, 0.2, 0.8])  # Example state for testing

# Calculate reward for each state-action pair and fill the reward table
for state_0 in range(state_size[0]):
    for state_1 in range(state_size[1]):
        for state_2 in range(state_size[2]):
            for state_3 in range(state_size[3]):
                for state_4 in range(state_size[4]):
                    for state_5 in range(state_size[5]):
                        # Construct a sample state based on the discretized index
                        state = [state_bins[i][index] for i, index in enumerate([state_0, state_1, state_2, state_3, state_4, state_5])]
                        # Calculate reward for each action
                        for action in range(action_size):
                            reward = calculate_reward(state, action)
                            reward_table[state_0, state_1, state_2, state_3, state_4, state_5, action] = reward

# Print the reward table
print("Reward Table (State-Action Rewards):")
for state_0 in range(state_size[0]):
    for state_1 in range(state_size[1]):
        for state_2 in range(state_size[2]):
            for state_3 in range(state_size[3]):
                for state_4 in range(state_size[4]):
                    for state_5 in range(state_size[5]):
                        print(f"State ({state_0}, {state_1}, {state_2}, {state_3}, {state_4}, {state_5}):")
                        for action in range(action_size):
                            print(f"  Action {action}: {reward_table[state_0, state_1, state_2, state_3, state_4, state_5, action]:.2f}")
```

```
Streaming output truncated to the last 5000 lines.
State (2, 3, 0, 1, 3, 2):
  Action 0: 0.33
  Action 1: 1.00
  Action 2: 0.67
State (2, 3, 0, 1, 3, 3):
  Action 0: 0.33
  Action 1: 1.00
  Action 2: 1.00
State (2, 3, 0, 2, 0, 0):
  Action 0: -2.67
  Action 1: -3.33
  Action 2: -3.33
State (2, 3, 0, 2, 0, 1):
  Action 0: -2.00
  Action 1: -2.67
  Action 2: -2.33
State (2, 3, 0, 2, 0, 2):
  Action 0: -1.33
  Action 1: -2.00
  Action 2: -1.33
State (2, 3, 0, 2, 0, 3):
  Action 0: -0.67
```

Hyperparameters

```python
[ ] # Hyperparameters

    # Controls how much new information overrides old information.
    # A high learning rate means that the agent places a lot of weight on new experiences, while a low learning rate means that the agent values older experiences more.
    # With a learning rate of 0.1, if the old Q-value is 5 and the new Q-value is calculated as 10, the updated Q-value will be 5 + 0.1 ×(10−5)= 5.5
    learning_rate = 0.1

    # determines the importance of future rewards.
    # A discount rate close to 1 means that future rewards are considered almost as important as immediate rewards, while a discount rate close to 0 means the agent values immediate rewards much more.
    # We put it as high as possible because future rewards are indeed significant in this environment
    discount_rate = 0.99

    # Controls the trade-off between exploration (choosing random actions) and exploitation (choosing the best-known action).
    # We put it as high as possible to allow the agent to learn as much as possible about the environment initially.
    exploration_rate = 1.0

    # The initial exploration rate.
    max_exploration_rate = 1.0

    # The minimum exploration rate that the agent can decay to. Ensures that the agent continues to explore at least a little throughout training.
    min_exploration_rate = 0.01

    # Determines how quickly the exploration rate decreases over time. A higher decay rate means that the exploration rate decreases faster.
    # Example: With a decay rate of 0.05, the exploration rate might decrease as follows over time: 1.0, 0.95, 0.90, etc.
    exploration_decay_rate = 0.05

    # The number of times the agent will run through the training loop.
    num_episodes = 5000

    # The maximum number of steps the agent can take within a single episode. This prevents the agent from getting stuck in an infinite loop.
    # In this case we set it to 1000, where the agent will reset after 1000 steps, even if it hasn't reached the goal.
    max_steps_per_episode = 1000
```

Store the total rewards accumulated by the agent in each episode during the training process.

```
[ ] # Training statistics
    rewards_per_episode = []
```

```
# Print the Q-Matrix (Sample)
print("\nSample Q-Matrix (first 3 states and all actions):")
for i in range(min(3, q_table.shape[0])):  # Print first 3 discretized states
    for action in range(action_space.n):
        print(f"State {i}, Action {action}: Q-value = {q_table[i, 0, 0, 0, 0, 0, action]:.2f}")
    print("-" * 30)
```

```
Sample Q-Matrix (first 3 states and all actions):
State 0, Action 0: Q-value = 0.00
State 0, Action 1: Q-value = 0.00
State 0, Action 2: Q-value = 0.00
------------------------------
State 1, Action 0: Q-value = 0.00
State 1, Action 1: Q-value = 0.00
State 1, Action 2: Q-value = 0.00
------------------------------
State 2, Action 0: Q-value = 0.00
State 2, Action 1: Q-value = 0.00
State 2, Action 2: Q-value = 0.00
------------------------------
```

```
[ ] # Generate and print a Reward Matrix (Sample)
    print("\nSample Reward Matrix (first 3 states and all actions):")
    for _ in range(3):  # Generate 3 random states
        state_sample = np.random.rand(6)  # Generate random continuous state
        for action in range(action_space.n):
            reward = calculate_reward(state_sample, action)
            print(f"State: {state_sample}, Action: {action}, Reward: {reward:.2f}")
        print("-" * 30)
```

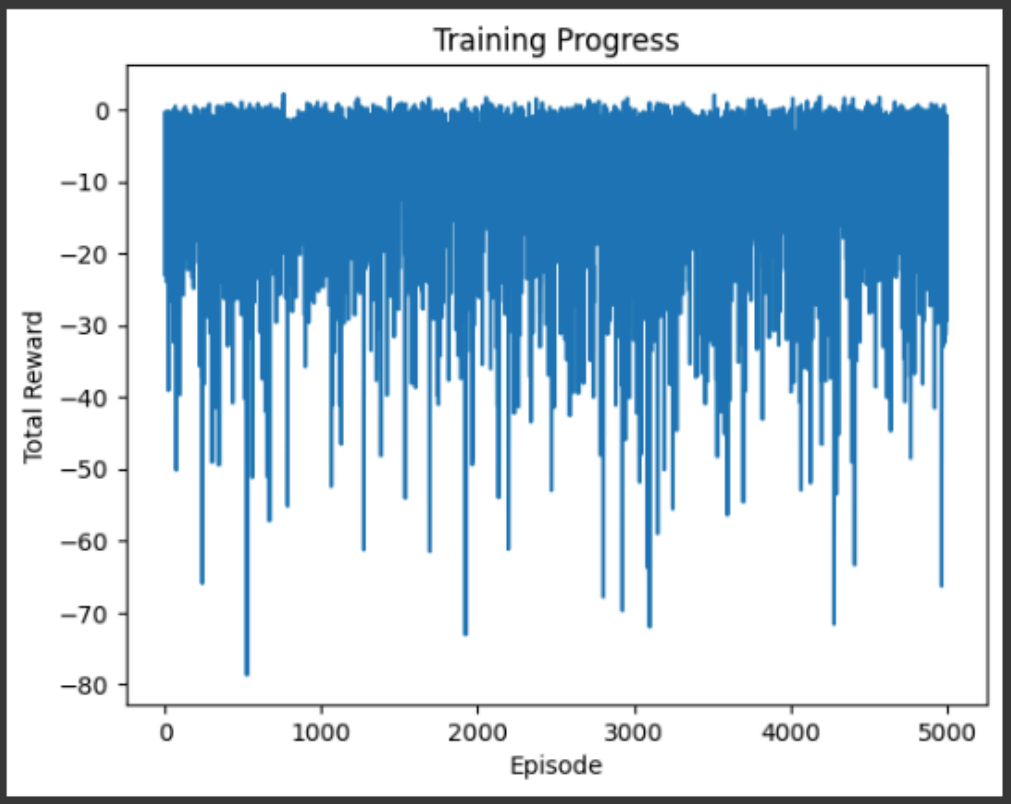
```
Sample Reward Matrix (first 3 states and all actions):
State: [0.42217384 0.14420366 0.73268431 0.74566005 0.20440887 0.28706438], Action: 0, Reward: -0.19
State: [0.42217384 0.14420366 0.73268431 0.74566005 0.20440887 0.28706438], Action: 1, Reward: -0.73
State: [0.42217384 0.14420366 0.73268431 0.74566005 0.20440887 0.28706438], Action: 2, Reward: -0.65
------------------------------
State: [0.49938211 0.64792558 0.53870154 0.72463123 0.27040506 0.49916379], Action: 0, Reward: 0.13
State: [0.49938211 0.64792558 0.53870154 0.72463123 0.27040506 0.49916379], Action: 1, Reward: -0.33
State: [0.49938211 0.64792558 0.53870154 0.72463123 0.27040506 0.49916379], Action: 2, Reward: -0.10
------------------------------
State: [0.90220183 0.35258867 0.30949096 0.92305487 0.91481852 0.89921237], Action: 0, Reward: -0.30
State: [0.90220183 0.35258867 0.30949096 0.92305487 0.91481852 0.89921237], Action: 1, Reward: -0.31
State: [0.90220183 0.35258867 0.30949096 0.92305487 0.91481852 0.89921237], Action: 2, Reward: -0.32
------------------------------
```

Q-Learning Algorithm

```python
# Q-Learning algorithm

for episode in range(num_episodes):
    state = reset_environment(observation_space)
    discrete_state = discretize_state(state, state_bins)
    done = False
    steps = 0
    total_reward = 0

    while not done and steps < max_steps_per_episode:
        steps += 1

        # Epsilon-greedy action selection
        exploration_rate_threshold = np.random.uniform(0, 1)
        if exploration_rate_threshold > exploration_rate:
            action = np.argmax(q_table[discrete_state])
        else:
            action = action_space.sample()

        # Take action and observe outcome
        new_state, reward, done = step_environment(state, action, observation_space)
        total_reward += reward
        discrete_new_state = discretize_state(new_state, state_bins)

        # Update Q-value
        old_value = q_table[discrete_state + (action,)]
        next_max = np.max(q_table[discrete_new_state])
        new_value = old_value + learning_rate * (reward + discount_rate * next_max - old_value)
        q_table[discrete_state + (action,)] = new_value

        # Update state
        state = new_state
        discrete_state = discrete_new_state

    # Decay exploration rate
    exploration_rate = min_exploration_rate + \
        (max_exploration_rate - min_exploration_rate) * np.exp(-exploration_decay_rate * episode)

    # Store episode reward
    rewards_per_episode.append(total_reward)

    # Print progress every 100 episodes
    if (episode + 1) % 100 == 0:
        avg_reward = np.mean(rewards_per_episode[-100:])
        print(f"Episode: {episode + 1}, Average Reward: {avg_reward:.2f}, Exploration Rate: {exploration_rate:.2f}")

print("Training completed!")
```

```
Episode: 100, Average Reward: -11.33, Exploration Rate: 0.02
Episode: 200, Average Reward: -7.43, Exploration Rate: 0.01
Episode: 300, Average Reward: -9.50, Exploration Rate: 0.01
Episode: 400, Average Reward: -8.45, Exploration Rate: 0.01
Episode: 500, Average Reward: -8.42, Exploration Rate: 0.01
Episode: 600, Average Reward: -9.88, Exploration Rate: 0.01
Episode: 700, Average Reward: -9.64, Exploration Rate: 0.01
Episode: 800, Average Reward: -8.02, Exploration Rate: 0.01
Episode: 900, Average Reward: -8.75, Exploration Rate: 0.01
Episode: 1000, Average Reward: -8.90, Exploration Rate: 0.01
Episode: 1100, Average Reward: -10.14, Exploration Rate: 0.01
Episode: 1200, Average Reward: -9.02, Exploration Rate: 0.01
Episode: 1300, Average Reward: -9.27, Exploration Rate: 0.01
Episode: 1400, Average Reward: -9.12, Exploration Rate: 0.01
Episode: 1500, Average Reward: -8.44, Exploration Rate: 0.01
Episode: 1600, Average Reward: -7.96, Exploration Rate: 0.01
Episode: 1700, Average Reward: -8.69, Exploration Rate: 0.01
Episode: 1800, Average Reward: -9.78, Exploration Rate: 0.01
Episode: 1900, Average Reward: -9.47, Exploration Rate: 0.01
Episode: 2000, Average Reward: -10.23, Exploration Rate: 0.01
Episode: 2100, Average Reward: -7.67, Exploration Rate: 0.01
Episode: 2200, Average Reward: -9.61, Exploration Rate: 0.01
Episode: 2300, Average Reward: -10.09, Exploration Rate: 0.01
Episode: 2400, Average Reward: -11.61, Exploration Rate: 0.01
Episode: 2500, Average Reward: -10.22, Exploration Rate: 0.01
Episode: 2600, Average Reward: -9.13, Exploration Rate: 0.01
Episode: 2700, Average Reward: -8.93, Exploration Rate: 0.01
Episode: 2800, Average Reward: -8.81, Exploration Rate: 0.01
Episode: 2900, Average Reward: -8.34, Exploration Rate: 0.01
Episode: 3000, Average Reward: -9.84, Exploration Rate: 0.01
Episode: 3100, Average Reward: -11.59, Exploration Rate: 0.01
Episode: 3200, Average Reward: -10.01, Exploration Rate: 0.01
Episode: 3300, Average Reward: -10.15, Exploration Rate: 0.01
Episode: 3400, Average Reward: -8.37, Exploration Rate: 0.01
Episode: 3500, Average Reward: -10.62, Exploration Rate: 0.01
Episode: 3600, Average Reward: -11.17, Exploration Rate: 0.01
Episode: 3700, Average Reward: -9.40, Exploration Rate: 0.01
Episode: 3800, Average Reward: -9.10, Exploration Rate: 0.01
Episode: 3900, Average Reward: -9.09, Exploration Rate: 0.01
Episode: 4000, Average Reward: -9.24, Exploration Rate: 0.01
Episode: 4100, Average Reward: -11.59, Exploration Rate: 0.01
Episode: 4200, Average Reward: -8.85, Exploration Rate: 0.01
Episode: 4300, Average Reward: -11.62, Exploration Rate: 0.01
Episode: 4400, Average Reward: -7.62, Exploration Rate: 0.01
Episode: 4500, Average Reward: -9.14, Exploration Rate: 0.01
Episode: 4600, Average Reward: -8.85, Exploration Rate: 0.01
Episode: 4700, Average Reward: -9.05, Exploration Rate: 0.01
Episode: 4800, Average Reward: -10.29, Exploration Rate: 0.01
Episode: 4900, Average Reward: -8.54, Exploration Rate: 0.01
Episode: 5000, Average Reward: -9.23, Exploration Rate: 0.01
Training completed!
```

```
[ ] # Optional: Plot the learning progress
    try:
        import matplotlib.pyplot as plt
        plt.plot(rewards_per_episode)
        plt.title('Training Progress')
        plt.xlabel('Episode')
        plt.ylabel('Total Reward')
        plt.show()
    except ImportError:
        print("matplotlib not available for plotting")
```



Second Example: Autonomous Warehouse Robot Navigation

Solution Proposal

Use Case Overview :

In a smart warehouse, robots are employed to pick and deliver items. The robot must navigate the warehouse efficiently, avoiding obstacles and minimizing travel time to maximize productivity.

Proposed Solution :

Use Q-Learning, a model-free reinforcement learning technique, to train the robot to navigate from its starting position to the target position while avoiding obstacles.

Solution Methodology:

- **Environment Representation:**
  - Represent the warehouse as a grid. Each cell in the grid corresponds to a state.
  - Obstacles are marked as inaccessible cells.
  - The robot starts at a defined cell and aims to reach a target cell.
- **Q-Learning Basics:**
  - States (S): All valid cells in the grid.
  - Actions (A): Move **up**, **down**, **left**, or **right**.
  - Rewards:
    - +100 for reaching the target cell.
    - -10 for hitting an obstacle.
    - -1 for each step to encourage faster navigation.
  - Q-Table: A table mapping state-action pairs to expected rewards.

- **Algorithm Steps:**
  - Initialize the Q-Table with zeros.
  - For each episode:

- Start from the initial state.
- Select an action using an epsilon-greedy policy.
- Perform the action and observe the reward and next state.
- Update the Q-value using the formula:

  $$Q(s, a) = Q(s, a) + \alpha * [r + \gamma * \max(Q(s', a')) — Q(s, a)]$$

  where α is the learning rate and γ is the discount factor.

- Repeat until the robot reaches the target or a maximum number of steps.

➢ Reduce epsilon over time to balance exploration and exploitation.

Description of Illustration :

<u>Warehouse Layout Representation :</u>

Grid Details: A 6x6 grid is depicted.

Accessible states are represented by black dots in cells.

Obstacles are marked with red dots at (2,2) and (3,1).

<u>Actions:</u>

Up (move to (x-1, y)),

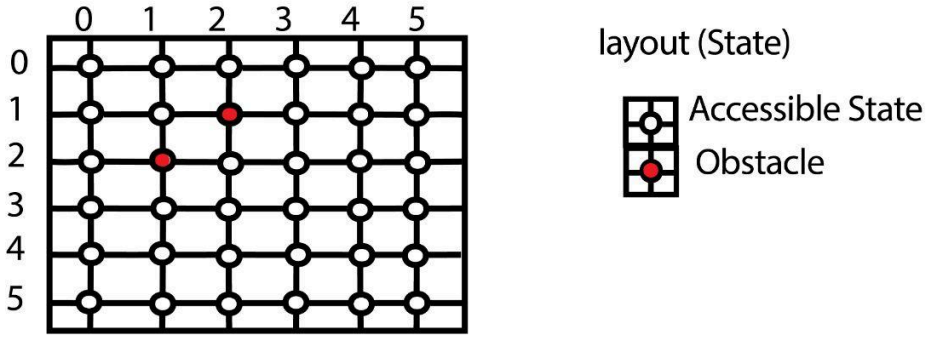
Left (move to (x, y-1)), Down (move to (x+1, y)), Right (move to (x, y+1)).

<u>Reward (R) Matrix :</u>

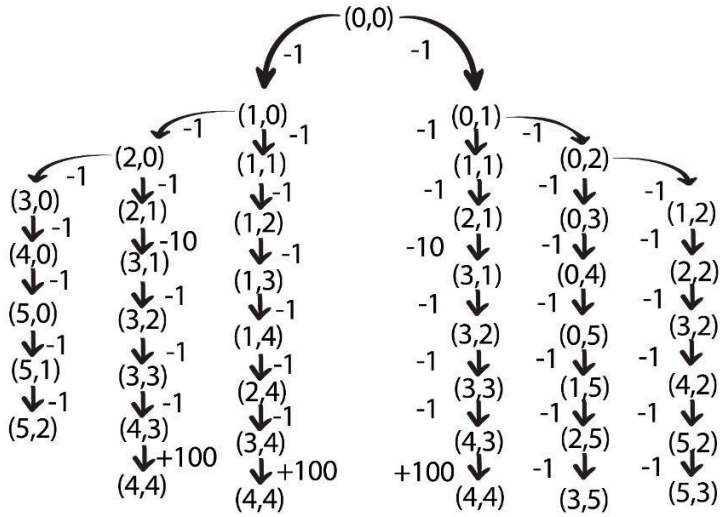
The R matrix is a 6x6 grid where:

I. +100 at the target cell (4,4).
II. -10 for obstacles (2,2) and (3,1).
III. -1 for all other accessible states (to encourage minimal steps).

Illustration :



Nodes-Diagram:



R matrix Action

$$R = \begin{array}{c} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{pmatrix} -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -10 & -1 & -1 & -1 \\ -1 & -10 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & 100 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 \end{pmatrix} \end{array}$$

Initialize Q matrix

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

Grid Start = (0,0)
Goal State = (4,4)
Rewards;- Target Cell (+100)
Obstacle (-10)
One step (-1)

Agent Navigation (Q-Learning Process)

The agent navigates the grid using Q-learning, with the following parameters:

Grid Start State: (0,0). Goal State: (4,4).

Rewards:

1. Reaching the target: +100.
2. Hitting an obstacle: -10.
3. Each step: -1.

State Transitions (illustrated as nodes):

Example Path: (0,0) → (0,1) → (1,1) → (2,1) → (3,1) (obstacle, -10) → …... → (4,4).

The agent uses exploration (epsilon-greedy) and exploitation to maximize cumulative rewards.

Commercialization Value and Stakeholders

Optimizing warehouse navigation (e.g., robots in a warehouse).

Commercialization Value:

1. Increased Efficiency: Reduces delivery time by optimizing robot paths and improves task throughput.
2. Scalability: Trained models can generalize to different warehouse layouts.
3. Cost Savings: Reduces energy consumption and machine wear-and-tear, and also minimizes operational costs (energy, wear and tear)..

Target Stakeholders

1. Warehouse Operators: Enhanced operational efficiency and reduced costs.
2. Logistics Companies: Improved order fulfillment times.
3. Robotics Manufacturers: Pretrained models increase product value and care for material handling.
4. E-commerce companies (Amazon, Flipkart) for warehouse logistics.

Justification:

By integrating Q-Learning into autonomous robots, warehouses can operate more effectively, with quicker ROI on robotic systems. This aligns with the growing demand for automation in logistics and e-commerce.

Autonomous Warehouse Robot Navigation Solution Code Explanations

All the libraries installation sources included

```
[ ] import numpy as np
    import random
```

Reinforced Learning Code

The grid is 6x6, and the parameters are as coded below:

```
[ ] # Define the grid and its parameters
    grid_size = 6
    start_state = (0, 0)
    goal_state = (4, 4)
    obstacles = [(2, 2), (3, 1)]
    actions = ['up', 'down', 'left', 'right']  # Define possible actions
```

Reward Matrix (R)

Default one step is -1 for every cell to encourage the optimal path If set to 0, it would produce many output with similar results.

Start state (0,0)

Goal state (4,4)

Obstacles (2,2) , (3,1)

```
[ ] # Reward matrix (R)
    R = np.full((grid_size, grid_size), -1)  # Default reward is -1 for every cell
    for x, y in obstacles:
        R[x, y] = -10  # Obstacles have a penalty
    R[goal_state] = 100  # Goal state has a high reward
```

Here's the reward matrix

```
[ ] # Print the Reward Matrix (R)
    print("Reward Matrix (R):")
    print(R)
```

```
Reward Matrix (R):
[[ -1  -1  -1  -1  -1  -1]
 [ -1  -1  -1  -1  -1  -1]
 [ -1  -1 -10  -1  -1  -1]
 [ -1 -10  -1  -1  -1  -1]
 [ -1  -1  -1  -1 100  -1]
 [ -1  -1  -1  -1  -1  -1]]
```

Initialization of Q matrix, which by default is all nil

```python
# Initialize Q-Table
Q = np.zeros((grid_size, grid_size, len(actions)))

# Print the initial Q Matrix
print("\nInitial Q Matrix:")
for x in range(grid_size):
    for y in range(grid_size):
        print(f"Q[{x},{y}] = {Q[x, y]}")
```

```
Initial Q Matrix:
Q[0,0] = [0. 0. 0. 0.]
Q[0,1] = [0. 0. 0. 0.]
Q[0,2] = [0. 0. 0. 0.]
Q[0,3] = [0. 0. 0. 0.]
Q[0,4] = [0. 0. 0. 0.]
Q[0,5] = [0. 0. 0. 0.]
Q[1,0] = [0. 0. 0. 0.]
Q[1,1] = [0. 0. 0. 0.]
Q[1,2] = [0. 0. 0. 0.]
Q[1,3] = [0. 0. 0. 0.]
Q[1,4] = [0. 0. 0. 0.]
Q[1,5] = [0. 0. 0. 0.]
Q[2,0] = [0. 0. 0. 0.]
Q[2,1] = [0. 0. 0. 0.]
Q[2,2] = [0. 0. 0. 0.]
Q[2,3] = [0. 0. 0. 0.]
Q[2,4] = [0. 0. 0. 0.]
Q[2,5] = [0. 0. 0. 0.]
Q[3,0] = [0. 0. 0. 0.]
Q[3,1] = [0. 0. 0. 0.]
```

```
Q[3,2] = [0. 0. 0. 0.]
Q[3,3] = [0. 0. 0. 0.]
Q[3,4] = [0. 0. 0. 0.]
Q[3,5] = [0. 0. 0. 0.]
Q[4,0] = [0. 0. 0. 0.]
Q[4,1] = [0. 0. 0. 0.]
Q[4,2] = [0. 0. 0. 0.]
Q[4,3] = [0. 0. 0. 0.]
Q[4,4] = [0. 0. 0. 0.]
Q[4,5] = [0. 0. 0. 0.]
Q[5,0] = [0. 0. 0. 0.]
Q[5,1] = [0. 0. 0. 0.]
Q[5,2] = [0. 0. 0. 0.]
Q[5,3] = [0. 0. 0. 0.]
Q[5,4] = [0. 0. 0. 0.]
Q[5,5] = [0. 0. 0. 0.]
```

Getting into it : Reinforced Learning Part

The setting of hyperparameters before truly diving into Reinforced Learning of Warehouse Robot Navigation

```
# Hyperparameters
alpha = 0.1  # Learning rate
gamma = 0.9  # Discount factor
epsilon = 0.2  # Epsilon-greedy exploration factor
episodes = 1000  # Number of training episodes
max_steps = 100  # Maximum steps per episode
```

The code to get the robot moving based on current state

```
# Function to get the next state based on the current action
def get_next_state(state, action):
    x, y = state
    if action == 'up' and x > 0:
        return (x - 1, y)
    elif action == 'down' and x < grid_size - 1:
        return (x + 1, y)
    elif action == 'left' and y > 0:
        return (x, y - 1)
    elif action == 'right' and y < grid_size - 1:
        return (x, y + 1)
    return state  # If the action is invalid, stay in the same state
```

The policy of robot choosing an action or where to move to

```
# Function to choose an action (epsilon-greedy policy)
def choose_action(state):
    if random.uniform(0, 1) < epsilon:
        return random.choice(range(len(actions)))  # Explore
    else:
        x, y = state
        return np.argmax(Q[x, y])  # Exploit
```

This is where it actually learn to move, could be said as training the "model"

```
[ ] # Q-Learning Algorithm
    for episode in range(episodes):
        state = start_state
        for _ in range(max_steps):
            # Choose an action
            action_index = choose_action(state)
            action = actions[action_index]

            # Take the action and observe the next state and reward
            next_state = get_next_state(state, action)
            x, y = state
            nx, ny = next_state
            reward = R[nx, ny]

            # Update Q-value
            Q[x, y, action_index] = Q[x, y, action_index] + alpha * (
                reward + gamma * np.max(Q[nx, ny]) - Q[x, y, action_index]
            )

            # Transition to the next state
            state = next_state

            # Break if the agent reaches the goal
            if state == goal_state:
                break
```

Explanation of Q

Final Q Table to show the value of all cells

Final Q-Table: The Q-values represent the learned utility of taking each action at each state. Higher values indicate better actions.

```
[ ] # Print the final Q-Table
    print("\nFinal Q-Table:")
    for x in range(grid_size):
        for y in range(grid_size):
            print(f"Q[{x},{y}] = {Q[x, y]}")
```

```
Final Q-Table:
Q[0,0] = [37.19341244 42.54907003 37.16879575 42.612659  ]
Q[0,1] = [42.46028474 48.45851    37.17811983 47.7889446 ]
Q[0,2] = [22.23422487 25.04161337 28.82484116 54.95327477]
Q[0,3] = [26.85703831 62.17098504 37.18099752 18.41597221]
Q[0,4] = [ 2.78756814 45.88840911  8.32691525 -0.81006028]
Q[0,5] = [-0.67078761 -0.63211996  5.28093609 -0.67934652]
Q[1,0] = [21.569732   10.20543034 16.66046732 48.45809564]
Q[1,1] = [42.39720737 41.12892311 41.88602348 54.9539    ]
Q[1,2] = [48.34770434 51.6818382  48.25460049 62.171     ]
Q[1,3] = [54.32407392 70.19       54.71923717 54.72951754]
Q[1,4] = [13.3780181  39.96672743 62.15641631  2.46904931]
Q[1,5] = [-0.37955169 -0.431146   21.33718563 -0.4900995 ]
Q[2,0] = [31.1687254  -1.56104131  0.90893137 -1.53468874]
```

```
Q[2,1] = [48.20206667 -0.97866319  1.11871715  8.85422792]
Q[2,2] = [22.27876454 70.17960482 11.86100553 28.59123377]
Q[2,3] = [61.99081568 79.1        52.19613383 78.72688253]
Q[2,4] = [25.68413582 88.99940955 23.99236531 11.59411808]
Q[2,5] = [1.30453061e+00 4.27047050e+01 7.48693850e+00 2.69018512e-02]
Q[3,0] = [ 1.38835391 -0.87141581 -1.04661746 -2.7368119 ]
Q[3,1] = [10.66443098  0.77670107 -0.52843008 42.19633775]
Q[3,2] = [20.37384486 21.17030877 12.86094151 79.09996543]
Q[3,3] = [70.04372578 88.3023595  69.91505269 89.        ]
Q[3,4] = [ 78.91819593 100.          79.05175081  77.72010302]
Q[3,5] = [ 3.74228741  0.629      88.83461859 31.73515089]
Q[4,0] = [-0.53855696 -0.70089197 -0.58519851  1.53068382]
Q[4,1] = [-1.909      -0.377929   -0.48691231 12.62212305]
Q[4,2] = [22.0362558  -0.19       -0.109      71.53369917]
Q[4,3] = [2.14451469e+01 9.14327900e-02 2.78557448e+01 9.99303801e+01]
Q[4,4] = [0. 0. 0. 0.]
Q[4,5] = [-0.1 -0.1 19.  -0.1]
Q[5,0] = [-0.50782752 -0.57655314 -0.58441256 -0.5454532 ]
Q[5,1] = [-0.2962     -0.29701    -0.44020834 -0.34487541]
Q[5,2] = [ 6.30837531 -0.199      -0.19981    -0.1       ]
Q[5,3] = [10.46541658 -0.1        -0.109       0.        ]
Q[5,4] = [0. 0. 0. 0.]
Q[5,5] = [-0.1  0.   0.   0. ]
```

Each cell in the Q-Table (e.g., Q [0,0]) contains an array of 4 Q-values corresponding to the 4 possible actions:

[Up, Down, Left, Right]

For example, in Q[0,0] = [37.26, 42.05, 37.24, 42.61]:

37.26 is the expected reward for moving Up from state (0,0)

42.05 is the expected reward for moving Down

37.24 is the expected reward for moving Left

42.61 is the expected reward for moving Right

These Q-values are learned over multiple episodes through exploration and exploitation.

Q[0,0] (Start State):

Q[0,0] = [37.26, 42.05, 37.24, 42.61]

Highest Q-value:

42.61 for action "Right"

This suggests the best action from (0,0) is to move Right to maximize the reward.

Q[4,4] (Goal State):

Q[4,4] = [0.0, 0.0, 0.0, 0.0]

All Q-values are 0 because the goal state gives an immediate reward of 100, and no further actions are possible from this state.

Q[2,2] (Obstacle State):

Q[2,2] = [14.70, 0.20, 9.04, 69.87]

Highest Q-value:

69.87 for action "Right"

This suggests the agent tries to escape the penalty by moving Right.

Q[2,4] (Near Goal):

Q[2,4] = [69.95, 89.00, 69.49, 68.78]

Highest Q-value:

89.00 for action "Down"

The agent prioritizes moving closer to the goal state.

Q Learning Algorithm Small Conclusion

The Q-learning algorithm enables the agent to learn an optimal policy:

Avoid Obstacles: Obstacles have a penalty (− 10 −10), so the agent learns to avoid states like (2,2) and (3,1).

Reach Goal: The goal state has a reward (100 100), so the agent learns to move toward (4,4).

Explore Efficient Paths: The agent doesn't necessarily take the shortest path but instead balances immediate rewards and future rewards to find an efficient path.

This is the agent's optimal path

```
[ ] # Test the agent's path from the start to the goal
    print("\nTesting the learned policy:")
    state = start_state
    path = [state]
    while state != goal_state:
        x, y = state
        action_index = np.argmax(Q[x, y])  # Choose the best action
        action = actions[action_index]
        state = get_next_state(state, action)
        path.append(state)

    # Print the optimal path
    print("Optimal Path taken by the agent:", path)
```

```
Testing the learned policy:
Optimal Path taken by the agent: [(0, 0), (0, 1), (1, 1), (1, 2), (1, 3), (2, 3), (3, 3), (3, 4), (4, 4)]
```

Conclusion:

The reward matrix R penalizes obstacles and encourages reaching the goal. The Q-table Q is initialized to zero for all states and actions.

The agent iteratively learns to navigate by updating the Q-values based on rewards and future expected values. Uses the epsilon-greedy policy to balance exploration and exploitation. Training:

Runs for 1000 episodes, with each episode allowing up to 100 steps. Stops when the agent reaches the goal. Testing:

After training, the agent uses the learned policy to find the optimal path from (0,0) to (4,4).

4. Future direction of Reinforcement Learning trends and evolution in the next five years in Malaysia industry and FIVE potential key companies in this field.

a) Future Direction of Reinforcement Learning in Malaysia

1. **Industry Adoption:**

-Industries such as logistics, healthcare and manufacturing will soon witness the deep penetration of RL in helping them make their decisions in real-time in dynamic environments.

-Predictive maintenance in manufacturing, autonomous logistics in supply chain management and etc will see more applications of this technology.

2. **Integration with IoT and Edge Computing:**

-Coming up with more IoT devices and always more RL algorithms will operate at the edge for real-time decision making without going through the cloud.

3. **Advancements in Autonomous Systems:**

-Most likely the development of autonomous vehicles and robotics in Malaysia will be influenced by RL-induced innovations in public transport and automated warehousing.

4. **Orientation Towards Sustainability:**

-Most likely, RL will find applications in optimization of energy management systems towards sundering wastage and promoting sustainability in industrial and residential applications.

5. **Education and Research Growth:**

More universities and institutions will start focusing on RL to build local capability while also increasing collaboration toward specific domain solutions with academia and research institutions.

b) Key Companies Driving RL in Malaysia

1. **Petronas:**

-Petronas will explore RL in predictive maintenance and optimize exploration and production processes in the oil and gas sector.

2. **DHL Malaysia:**

-DHL Malaysia has utilized reinforcement learning for optimizing logistics, plan route paths, and help autonomous warehouse operations.

3. **Malaysia Airlines**:

-Malaysia Airlines is utilizing RL for revenue management, flight scheduling, and fuel optimization.

4. **Grab**:

-Ride and delivery platforms of Grab will incorporate dynamic pricing, route optimization, and personalized offer service enhancement using RL.

5. **Fusionex**:

-Fusionex is a Malaysian tech company specializing in creating AI-based solutions which including RL applications.